MINI PROJECT REPORT



Machine Learning-Based Network Intrusion Detection on the UNSW-NB15 Dataset

A Comparison of Logistic Regression, Decision Tree and Random Forest

Subject: Machine Learning

Submitted by
Sontakke Sayali Premraj
(MIS: 712652070)
M.Tech First Year (2026-27), Data Science

Submitted to
Dr. Yashodhara V. Haribhakta
(4105002001)
COEP Technological University, Pune

November 2026

Declaration

I declare that this mini project report titled "Machine Learning-Based Network Intrusion Detection on the UNSW-NB15 Dataset" is the result of my own work carried out for the Machine Learning course. All experiments were run by me on the publicly available UNSW-NB15 dataset using the code described in this report. Sources of information, datasets and software libraries are acknowledged in the References. Results are reported as obtained from my own runs and have not been copied from published papers.

Sontakke Sayali Premraj
MIS Number: 712652070
Date: 08/11/2026

Certified that the above statement made by the student is correct to the best of my knowledge and belief.

Dr. Yashodhara V. Haribhakta
(COEP Technological University, Pune)

Acknowledgement

I sincerely thank Dr. Yashodhara V. Haribhakta, my Machine Learning faculty, for her guidance, for the mini project assignment and for the questions that helped me explain the project more clearly. I also thank the Cyber Range Lab of the Australian Centre for Cyber Security at UNSW Canberra for making the UNSW-NB15 dataset freely available for academic research, and the authors of the research papers reviewed in this report.

Sontakke Sayali Premraj
(712652070)

Abstract

Networks carry both normal and malicious traffic, and rules written by hand can only detect attacks that someone has already described. This project studies whether supervised Machine Learning can learn to separate normal traffic from attack traffic using labelled network-flow records. The public UNSW-NB15 dataset, created at UNSW Canberra, was used through its official training file (175,341 records) and testing file (82,332 records). Each record is one network connection described by 42 measured columns (features), and the dataset supplies the correct answer in a separate label column (0 normal, 1 attack).

The report explains how the learning works: during training the model sees the features together with the correct answer and learns patterns; during prediction it sees only the 42 features and outputs a probability of attack. No single column decides the result. The model combines many columns, and the most influential ones in the Random Forest were the time-to-live of source packets (sttl) and the connection-state count (ct_state_ttl) by one measure, and the service, a connection-count column and the source byte count by another.

Logistic Regression, Decision Tree and Random Forest were compared with a majority-class baseline. Preprocessing was placed in a pipeline fitted on training data only, hyperparameters were tuned with 5-fold stratified cross-validation, and each final model was tested once on the official test file. The baseline reached an accuracy of 0.5506. Random Forest performed best overall (accuracy 0.8719, F1-score 0.8944, recall 0.9853) and missed 668 of 45,332 attacks. The main weakness of all models was false alarms on normal traffic (24.1% to 38.7%). The results come from one laboratory dataset generated in 2015 and should not be taken as proof of performance on real networks.

Keywords: intrusion detection, machine learning, UNSW-NB15, feature importance, Random Forest, Decision Tree, Logistic Regression, false-positive rate.

Table of Contents

Abstract **3**
1. Introduction **7**
1.1 Background 7
1.2 Motivation 7
1.3 Problem Statement 7
1.4 Scope of the Project 7
1.5 Organisation of the Report 7
2. Objectives **8**
2.1 Aim 8
2.2 Research Question 8
2.3 Specific Objectives 8
2.4 Sub-questions 8
2.5 Success Criteria 8
3. Literature Review **9**
3.1 Paper 1: Creation of the UNSW-NB15 Dataset 9
3.2 Paper 2: Feature Selection with Several Classifiers 9
3.3 Paper 3: Feature Selection versus Feature Extraction 9
3.4 Paper 4: Generalisation across Datasets 9
3.5 Paper 5: Same-Dataset versus Cross-Dataset Performance 9
3.6 Comparison of the Reviewed Papers 10
3.7 Summary of the Literature and Positioning of This Project 10
4. Dataset **10**
4.1 Source and Creation 10
4.2 Files Used 10
4.3 Columns and Their Roles 10
4.4 Target Variable 11
4.5 Attack Categories 11
4.6 Actual Records from the Dataset 11
4.7 Class Distribution 12
4.8 Data Quality 12
4.9 Licence and Citation 13
5. Why This Dataset Was Selected **13**
5.1 Reasons for Selecting UNSW-NB15 13
5.2 Comparison with the Alternatives 13
6. Learning Process and Models **14**
6.1 Supervised Learning Formulation 14
6.2 Two Phases: Training and Prediction 14
6.3 What "Learning" Means for Each Model 15
6.4 From Probability to Decision 16
7. Feature Roles and Importance Analysis **16**
7.1 The Role of Every Column 16
7.2 What the 42 Feature Columns Measure 17
7.3 A Worked Comparison of Two Real Records 17
7.4 Evidence That No Single Column Decides 18
7.5 The Most Influential Columns 19
7.6 Limitations of the Importance Analysis 19
8. Research Methodology **20**
8.1 Research Design 20
8.2 Phases of the Study 20
8.3 Data Collection 20
8.4 Variables 20
8.5 Model Selection Approach 21
8.6 Evaluation Approach 21
8.7 Validity and Reliability 21
8.8 Tools and Environment 21
8.9 Ethical Considerations 21

9. System Design **21**
9.1 Overview 21
9.2 Components 22
9.3 Input and Output Specification 22
9.4 Design Decisions 23
10. Experimentation Design **23**
10.1 Use of the Data 23
10.2 Experimental Procedure 23
10.3 List of Experiments 24
10.4 Controls and Fairness 24
11. Parameters **24**
11.1 General Settings 24
11.2 Preprocessing Parameters 25
11.3 Hyperparameters Searched and Chosen 25
11.4 Role of the Main Hyperparameters 25
11.5 Evaluation Metrics and Formulas 26
12. Implementation and Evidence **26**
12.1 Implementation Overview 26
12.2 Libraries and Main Components 26
12.3 Key Code Excerpts 27
12.4 Using the Trained Model: Input to Output 27
12.5 Implementation Proof Cell 27
12.6 Reproducibility 28
13. Data Analysis **28**
13.1 Class Balance 28
13.2 Differences between the Training and Testing Files 29
13.3 Columns Most Associated with the Label 29
14. Results **30**
14.1 Overall Model Comparison 30
14.2 Confusion Matrices 31
14.3 Detection by Attack Category 31
14.4 Overfitting Check and Random-Split Contrast 32
14.5 Feature Importance 32
15. Discussion and Findings **32**
15.1 Findings 32
15.2 Answers to the Sub-questions 33
15.3 Relation to the Literature 34
16. Limitations **34**
16.1 Data 34
16.2 Method 34
16.3 Results and Generalisation 34
17. Conclusion **35**
18. Future Work **35**
References **35**
Appendix A: Feature Dictionary **37**
Appendix B: Notebook Structure and Reproduction **38**

List of Tables

Table 1: Comparison of the five reviewed papers 10
Table 2: Files and size (confirmed from the project run) 10
Table 3: Attack categories in UNSW-NB15 (paraphrased from the dataset description) 11
Table 4: Sample of real records from the training file 12
Table 5: Class distribution in the training and testing files (from the project run) 12
Table 6: Data quality findings (from the project run) 12
Table 7: Comparison of candidate datasets 13
Table 8: What the model sees in each phase 14

Table 9: Role of each type of column 16
Table 10: The six groups of feature columns 17
Table 11: A normal record and a DoS attack record from the training file 17
Table 12: Columns that influence the Random Forest most, with the evidence 19
Table 13: Phases of the research methodology 20
Table 14: Tools and environment 21
Table 15: Components of the system 22
Table 16: Main design decisions and their reasons 23
Table 17: How each file was used 23
Table 18: Experiments carried out 24
Table 19: General experiment settings 24
Table 20: Preprocessing parameters 25
Table 21: Hyperparameter search grids and the values chosen by cross-validation 25
Table 22: Tuning results and time 25
Table 23: Evaluation metrics 26
Table 24: Libraries and classes used 26
Table 25: Two ways of giving the input 27
Table 26: What the proof cell checks 28
Table 27: Numeric features with the strongest correlation with the label (training file) 29
Table 28: Performance of all models on the official test file 30
Table 29: Confusion counts on the test file 31
Table 30: Detection rate per category on the test file (for Normal: share correctly passed) 31
Table 31: Training-side and test scores (F1-score) 32
Table 32: Answers to the research sub-questions 33
Table 33: Feature dictionary 37
Table 34: Main parts of the notebook 38

List of Figures

Figure 1: Role of each column in the dataset files 11
Figure 2: How the model learns (Phase 1) and how it is used (Phase 2) 14
Figure 3: A tiny illustrative decision tree (invented thresholds; the real trees are much deeper) 16
Figure 4: Random Forest impurity-based importance of the top 15 columns 18
Figure 5: Random Forest permutation importance of the top 15 columns (drop in F1-score on a test sample) 19
Figure 6: System design of the project in six layers 22
Figure 7: Normal and attack records in the training and testing files 29
Figure 8: Share of each attack category in the training and testing files 29
Figure 9: Key test-file metrics for the three models (lower is better for FPR and FNR) 30
Figure 10: Confusion matrices on the official test file (rows are the true class, columns the predicted class) 31
Figure 11: Detection rate per category (darker means higher) 32

1. Introduction

1.1 Background

Computer networks are used for banking, education, healthcare and everyday communication, and they are attacked constantly. An Intrusion Detection System (IDS) monitors network activity and raises an alert when it sees behaviour that looks malicious. Traditional detection relies on signatures, which are fixed patterns describing known attacks. Signatures are precise for known attacks, but they miss new attacks and small variations of old ones.

Machine Learning offers a different approach. Instead of writing rules by hand, a model is shown many labelled examples of normal and attack traffic and learns which measured characteristics, such as packet counts, byte counts, duration and time-to-live values, separate the two.

1.2 Motivation

I selected intrusion detection because it connects my cybersecurity background with Machine Learning. In cybersecurity I learned how attacks are normally detected with rules. This project lets me test whether a model can learn the same distinction from data and, just as importantly, understand where such a model fails. Every record in the dataset has a known label, so the results can be measured exactly.

1.3 Problem Statement

Signature-based detection cannot recognise attacks it has not been told about. The problem addressed here is to classify a single network connection as either normal or attack, using only the measured features of that connection, and to measure honestly how reliable such a classifier is, including how many false alarms it raises and which attacks it misses.

1.4 Scope of the Project

- **Included:** one public dataset (UNSW-NB15), the official training and testing files, binary classification, three standard models plus a baseline, a leakage-safe pipeline, an evaluation with several metrics, an error analysis by attack category, and an explanation of which columns influence the decision.
- **Excluded:** the design of a new algorithm, multiclass classification as a main task, feature selection, testing on other datasets, and the building of a working security product.

1.5 Organisation of the Report

The remainder of the report is organised as follows. Chapter 2 states the objectives and Chapter 3 reviews the literature. Chapters 4 and 5 describe the dataset and the reasons for selecting it. Chapters 6 and 7 describe the learning process and the role of the feature columns. Chapters 8 to 11 present the methodology, system design, experimental design and parameters, and Chapter 12 documents the implementation. Chapters 13 to 15 give the data analysis, results and discussion. Chapters 16 to 18 give the limitations, conclusion and future work. The appendices contain the feature dictionary and the notebook structure.

2. Objectives

2.1 Aim

To measure how well supervised Machine Learning models can separate normal network traffic from attack traffic on the UNSW-NB15 dataset, to explain how the models reach their decisions, and to identify where they fail.

2.2 Research Question

How well do Logistic Regression, Decision Tree and Random Forest classify traffic as normal or attack on the official UNSW-NB15 training and testing files, which columns influence their decisions, and how do false alarms and missed attacks differ across attack types?

2.3 Specific Objectives

1. To inspect the real dataset: its shape, columns, labels, missing values, duplicate records and class balance.
2. To build a preprocessing pipeline that does not leak information from the test data into training.
3. To train three models and a baseline under identical conditions.
4. To evaluate the models with metrics that matter for security, namely precision, recall, false-positive rate and false-negative rate, and not accuracy alone.
5. To analyse errors by attack category.
6. To explain how the models learn, and which feature columns influence their decisions and by how much.
7. To state the limitations of the experiment honestly.

2.4 Sub-questions

- **SQ1:** Do all three models perform better than a baseline that always predicts the majority class?
- **SQ2:** Which model gives the best balance of detection and false alarms on the official test file?
- **SQ3:** Which attack categories are hardest to detect?
- **SQ4:** Does a random split inside the training file give a more optimistic result than the official test file?
- **SQ5:** Which columns influence the decision most, and does any single column decide alone?

2.5 Success Criteria

The project is considered successful if every model is compared with the baseline, the test file is used only once for the final evaluation, no information from the test file is used to fit or tune any step, all reported numbers come from the actual run, the decision process is explained clearly, and the limitations are stated. No target accuracy was fixed in advance, because a threshold chosen without evidence would be arbitrary.

3. Literature Review

Five research papers were selected because they use UNSW-NB15 or closely related methods and cover the questions that matter for this project: how the dataset was built, how classifiers compare, how rare classes behave and whether results transfer to other data. Three of the five are preprints posted on arXiv and were not verified as peer-reviewed publications.

3.1 Paper 1: Creation of the UNSW-NB15 Dataset

Moustafa and Slay [1] introduced UNSW-NB15. They argued that older benchmarks such as KDD98, KDDCUP99 and NSL-KDD are about a decade old and do not reflect modern traffic or modern low-footprint attacks. The new dataset mixes real modern normal activity with synthetic attack behaviour generated in a laboratory. This paper does not compare classifiers. It is cited because the dataset owners require it and because it explains why the dataset exists.

3.2 Paper 2: Feature Selection with Several Classifiers

Kasongo and Sun [2] used the official training and testing split. They reduced the feature set with an XGBoost-based method and tested SVM, k-nearest neighbours, Logistic Regression, an artificial neural network and a Decision Tree. For binary classification, the test accuracy of the Decision Tree rose from 88.13% to 90.85% with the reduced feature set. They also observed that one feature, sttl, had by far the highest importance, and that training accuracy was clearly higher than test accuracy for several models.

3.3 Paper 3: Feature Selection versus Feature Extraction

Ngo, Vuong, Luong and Tran [3] (arXiv preprint, 2023) compared correlation-based feature selection with Principal Component Analysis on the official split, using Decision Tree, Random Forest, k-nearest neighbours, a multilayer perceptron and Naive Bayes for binary and multiclass tasks. In binary classification the normal class was detected less well than the attack class. In multiclass classification the Analysis and Backdoor categories were almost never recognised.

3.4 Paper 4: Generalisation across Datasets

Layeghy and Portmann [4] (arXiv preprint, 2022) trained seven supervised and unsupervised models on NetFlow-format versions of four benchmark datasets. No model generalised across all datasets, and results were asymmetric when the source and target datasets were swapped. They used SHAP analysis to show that a feature strongly linked to the label in one dataset can lose that link in another.

3.5 Paper 5: Same-Dataset versus Cross-Dataset Performance

Hossain, Khan, Islam, Islam and Gedeon [5] (arXiv preprint, 2026) tested Random Forest, Logistic Regression and Naive Bayes on UNSW-NB15 and TON_IoT. Random Forest reached 95.08% accuracy on UNSW-NB15 within the same dataset, using a random 70/30 split of their own cleaned data of 257,673 records, but its accuracy fell below 40% when trained on one dataset and tested on the other.

3.6 Comparison of the Reviewed Papers

Table 1: Comparison of the five reviewed papers

No.	Paper	Data and split	Models	Main point
1	Moustafa and Slay (2015)	UNSW-NB15 (creation)	None	Origin and design of the dataset
2	Kasongo and Sun (2020)	Official split	SVM, kNN, LR, ANN, DT	Feature selection helps; train/test gap
3	Ngo et al. (2023)	Official split	DT, RF, kNN, MLP, NB	Rare classes are missed
4	Layeghy and Portmann (2022)	NetFlow versions of four datasets	Seven models	Poor generalisation across datasets
5	Hossain et al. (2026)	Own random split, plus TON_IoT	RF, LR, NB	High same-dataset score, low cross-dataset score

3.7 Summary of the Literature and Positioning of This Project

Previous researchers have used UNSW-NB15 mainly to compare classifiers and to reduce the number of features. Three problems appear repeatedly: rare attack classes are hard to detect, reported scores depend strongly on how the data is split, and high benchmark scores often do not transfer to other datasets. This project does not claim a new algorithm. It reproduces a small part of the earlier work (a comparison of standard models on the official split) and focuses on an honest evaluation and on explaining how the models decide.

4. Dataset

4.1 Source and Creation

UNSW-NB15 was created by the Cyber Range Lab of the Australian Centre for Cyber Security at UNSW Canberra [1]. Raw network packets were generated with the IXIA PerfectStorm tool, which mixes real modern normal activity with synthetic attack behaviour, and tools were then used to extract one record per network connection (flow). The official page is https://research.unsw.edu.au/projects/unsw-nb15-dataset. The dataset is free for academic research and users are asked to cite the authors' papers.

4.2 Files Used

The complete dataset contains about 2.54 million records stored in four raw files without a header row. This project uses the two partitioned files that are widely used in research: **UNSW_NB15_training-set.csv** and **UNSW_NB15_testing-set.csv**. Both files were loaded successfully in Google Colab.

Table 2: Files and size (confirmed from the project run)

File	Rows	Columns	Role
UNSW_NB15_training-set.csv	175,341	45	Learning and tuning
UNSW_NB15_testing-set.csv	82,332	45	Final evaluation only

4.3 Columns and Their Roles

Each file has 45 columns. Chapter 7 explains the role of each column in detail. In short, 42 columns are input features, one is an identifier (id), one names the attack type (attack_cat) and one is the

correct answer (label). Of the 42 features, 39 are numeric and 3 are categorical (proto, service and state). A feature dictionary is given in Appendix A.

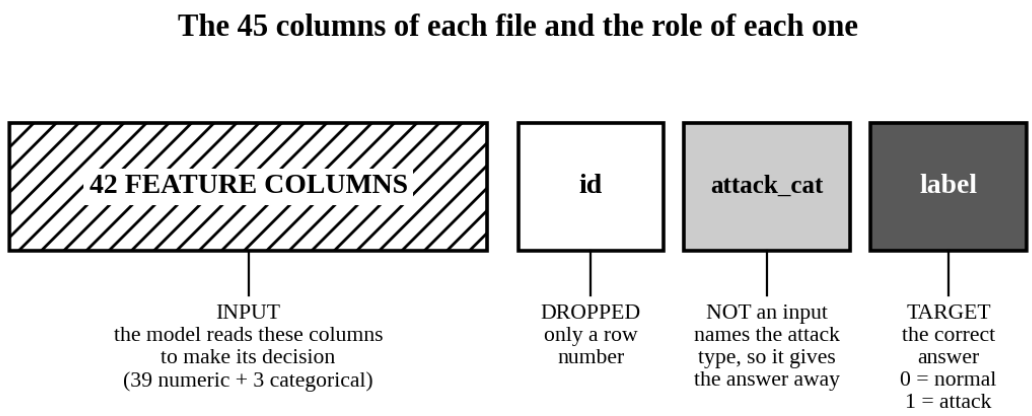


Figure 1: Role of each column in the dataset files

4.4 Target Variable

The target is the column **label**, where 0 means normal and 1 means attack. This makes the task a binary classification problem. The column **attack_cat** names the type of traffic (Normal or one of nine attack categories). It determines the label exactly, as shown by the cross-tabulation in the project run, so it was not used as an input feature: it would reveal the answer (data leakage). It was used only after prediction to analyse errors.

4.5 Attack Categories

Table 3: Attack categories in UNSW-NB15 (paraphrased from the dataset description)

Category	Meaning
Normal	Ordinary, legitimate traffic
Generic	A technique that works against block ciphers regardless of how they are built
Exploits	Use of a bug or vulnerability in software to cause unintended behaviour
Fuzzers	Feeding random or malformed data to try to crash or hang a program or service
DoS	Attempts to make a service or network resource unavailable
Reconnaissance	Gathering information about a target before an attack, such as port scanning
Analysis	Port scans, spam and web-page intrusion probes grouped together
Backdoor	A hidden way to bypass normal authentication and obtain remote access
Shellcode	A small payload that opens a command shell on the victim machine
Worms	Self-copying attacks that spread to other computers

4.6 Actual Records from the Dataset

The table below shows real records taken from the training file during the project run (values are copied from the notebook output). The DoS and Generic attack records have a very short duration,

two packets sent and none received, and a sttl value of 254, while the normal TCP records show packets in both directions.

Table 4: Sample of real records from the training file

id	dur	proto	service	state	spkts	dpkts	sbytes	dbytes	sttl	attack_cat	label
95745	0.661656	tcp	http	FIN	10	10	804	1330	62	Normal	0
44164	1.501572	tcp	http	FIN	60	16	68195	698	254	Normal	0
13714	0.333839	tcp	ssh	FIN	32	24	3728	5474	31	Normal	0
48018	0.675470	tcp	ftp	FIN	12	12	2616	682	254	Fuzzers	1
48933	0.505999	tcp	http	FIN	12	26	860	26136	62	Exploits	1
49000	0.000009	encap	-	INT	2	0	200	0	254	DoS	1
117205	0.000003	udp	dns	INT	2	0	114	0	254	Generic	1

4.7 Class Distribution

Table 5: Class distribution in the training and testing files (from the project run)

Class	Training count	Training %	Testing count	Testing %
Normal (label 0)	56,000	31.94	37,000	44.94
Attack (label 1)	119,341	68.06	45,332	55.06
Generic	40,000	22.81	18,871	22.92
Exploits	33,393	19.04	11,132	13.52
Fuzzers	18,184	10.37	6,062	7.36
DoS	12,264	6.99	4,089	4.97
Reconnaissance	10,491	5.98	3,496	4.25
Analysis	2,000	1.14	677	0.82
Backdoor	1,746	1.00	583	0.71
Shellcode	1,133	0.65	378	0.46
Worms	130	0.07	44	0.05

4.8 Data Quality

Table 6: Data quality findings (from the project run)

Check	Training file	Testing file
Missing cells	0	0
Infinite cells	0	0
Duplicate rows (ignoring id and attack_cat)	74,072 (about 42.2%)	28,380 (about 34.5%)
Test rows with features identical to a training row	Not applicable	8,541 (10.37%)
Distinct proto / service / state values	133 / 13 / 9	131 / 13 / 7
Categories present in test but not in training	Not applicable	state: ACC, CLO

The data has no missing or infinite values, but it contains a large share of duplicate rows, and about one tenth of the test records have exactly the same feature values as some training record. These facts matter for the interpretation of the results (see Chapter 15).

4.9 Licence and Citation

UNSW-NB15 may be used freely for academic research provided the dataset creators' papers are cited. The primary citation is Moustafa and Slay (2015) [1], and a second paper by the same authors is listed in the References [6].

5. Why This Dataset Was Selected

Before choosing, I compared three candidate datasets for network intrusion detection: UNSW-NB15, CIC-IDS2017 and CSE-CIC-IDS2018. I verified information about UNSW-NB15 and CIC-IDS2017 from their official pages and from research papers. I did not examine CSE-CIC-IDS2018 in detail, so it is neither recommended nor rejected here.

5.1 Reasons for Selecting UNSW-NB15

1. **Relevance.** It is a network intrusion dataset with normal traffic and nine documented attack types, so it matches the problem statement directly.
2. **Reliable labels.** Every record has a binary label and an attack category, which makes supervised learning and error analysis by category possible.
3. **A ready-made official split.** The partitioned training and testing files are the standard setup in the literature. Using them avoids inventing my own split, keeps the test data separate and allows a fair comparison of patterns with published work.
4. **Suitable size.** The two files together hold about 257,000 records, which fits the memory and time limits of free Google Colab. CIC-IDS2017 is reported with about 3.1 million instances.
5. **Research coverage.** Four of the five papers reviewed use UNSW-NB15 or a version of it, so results can be discussed against real literature.
6. **Mixed feature types.** Numeric and categorical features allow a realistic pipeline with imputation, scaling and encoding.
7. **Documentation and licence.** There is an official description, a feature list, a free academic licence and a clear citation requirement.
8. **Ease of explanation.** Features such as packets, bytes, duration and time-to-live can be explained in simple words.

5.2 Comparison with the Alternatives

Table 7: Comparison of candidate datasets

Criterion	UNSW-NB15	CIC-IDS2017
Source	UNSW Canberra	Canadian Institute for Cybersecurity, UNB
Size	About 2.54 million records in full; 257,673 in two partitioned files	About 3.1 million instances reported in one paper
Features	49 in the full data; 42 usable inputs in the partitioned files	80 or more flow features (83 reported in one paper)
Official train/test files	Yes	Per-day files; no single standard split
Fits free Colab easily	Yes	Harder, because of its size

Criterion	UNSW-NB15	CIC-IDS2017
Known concerns	Many duplicates; 2015 laboratory data	Strong class imbalance; one paper reports data-quality problems and built a corrected version

6. Learning Process and Models

This chapter formulates the task as supervised learning and describes the three classifiers that were compared.

6.1 Supervised Learning Formulation

In supervised learning a model is fitted to labelled examples (x, y), where x is the vector of 42 measured features of one network flow and y is its label (0 normal, 1 attack). The model uses the 175,341 records of the training file to learn a function f(x) that estimates the probability of attack, and its generalisation is then assessed on records that were not used for fitting.

6.2 Two Phases: Training and Prediction

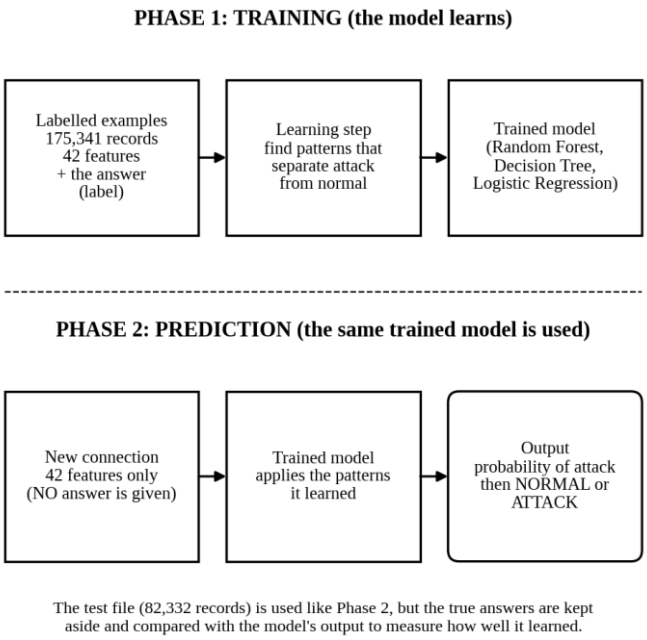


Figure 2: How the model learns (Phase 1) and how it is used (Phase 2)

Table 8: What the model sees in each phase

Phase	Input given to the model	Is the correct answer given?	Result
Training (training file)	42 features plus the label of each record	Yes, so the model can learn from it	A trained model
Prediction on new data	42 features only	No	Probability of attack, then NORMAL or ATTACK

Phase	Input given to the model	Is the correct answer given?	Result
Testing (testing file)	42 features only	No. The true answers are kept aside and compared afterwards	Metrics that show how well the model learned

At prediction time the model receives only the 42 features. The label column and the attack_cat column are therefore excluded from the input.

6.3 What "Learning" Means for Each Model

The three models are trained on the same data but represent what they learn differently.

Logistic Regression gives every feature a weight and adds them up. First it calculates a score z = b0 + b1 x f1 + b2 x f2 + ... where f1, f2 and so on are the feature values and b1, b2 and so on are the weights learned from the training data. Then the sigmoid function turns the score into a probability of attack: p = 1 / (1 + e^(-z)). A large positive score gives a probability near 1 (attack) and a large negative score gives a probability near 0 (normal).

Decision Tree learns a chain of yes/no questions about the features. At each step it chooses the question that separates attack records from normal records best. A common measure is Gini impurity = 1 - (share of attack squared + share of normal squared); a value of 0 means a perfectly pure group. In the illustrative example below, a group of 100 connections with 60 attacks and 40 normal records has a Gini impurity of 1 - (0.36 + 0.16) = 0.48. A question about sttl splits it into a group of 50 (48 attacks, 2 normal; impurity 0.0768) and a group of 50 (12 attacks, 38 normal; impurity 0.3648). The weighted impurity after the split is 0.5 x 0.0768 + 0.5 x 0.3648 = 0.2208, which is lower than 0.48, so the question is useful and is kept. The tree repeats this until it reaches leaves that give a final answer.

Illustrative decision tree (invented thresholds, NOT the real trained tree)

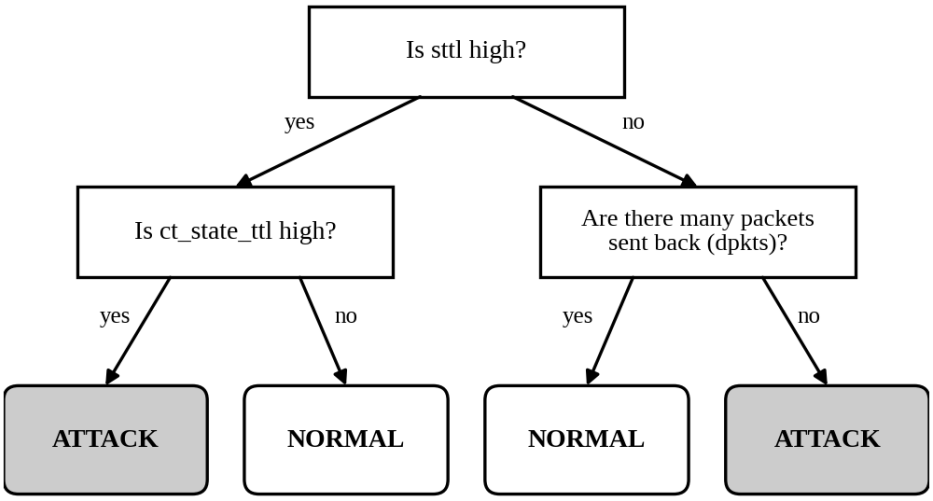


Figure 3: A tiny illustrative decision tree (invented thresholds; the real trees are much deeper)

Random Forest builds many decision trees (100 in this project). Each tree is trained on a random sample of the training rows and considers a random subset of the features at each split, so the trees

make different mistakes. To decide about a new connection, all trees give their answer and the results are combined. For example, if 87 of 100 trees say attack, the probability of attack is about 0.87 and the final answer is ATTACK (illustrative numbers; the library actually averages the probability estimates of the trees).

6.4 From Probability to Decision

Every model produces a probability of attack. A threshold turns it into a decision: if the probability is 0.5 or higher the decision is ATTACK, otherwise NORMAL. The threshold 0.5 is the default and was not tuned in this project. A different threshold would trade false alarms against missed attacks.

7. Feature Roles and Importance Analysis

This chapter describes the role of every column in the dataset files, the six groups of feature columns, and the evidence on which features influence the Random Forest. The evidence shows that no single column decides the outcome; the models combine all 42 features.

7.1 The Role of Every Column

Table 9: Role of each type of column

Column(s)	Role in the project	Used by the model when predicting?
42 features (dur, proto, service, state, spkts, dpkts, sbytes, dbytes, sttl, dttl, ct_state_ttl and others)	Measurements that describe one connection. They are the input	Yes
id	Only a row number. It says nothing about the traffic	No (dropped)
attack_cat	Names the attack type. It reveals the answer, so using it would be cheating (data leakage)	No (excluded). Used afterwards to analyse errors by attack type
label	The correct answer: 0 normal, 1 attack. It is the target	No. Used in training so the model can learn, and afterwards to check the answers

7.2 What the 42 Feature Columns Measure

The features fall into six groups. The last column of the table gives a tendency that may help to separate attacks from normal traffic; these are not fixed rules, because the models learn from the data which tendencies are useful.

Table 10: The six groups of feature columns

Group	Columns	What they measure	Why they may help
Size and duration (8)	dur, spkts, dpkts, sbytes, dbytes, smean, dmean, response_body_len	How long the connection lasted, and how many packets and bytes each side sent	Floods and scans often use very short connections and tiny packets

Group	Columns	What they measure	Why they may help
Time to live (2)	sttl, dttl	A counter in each packet; starting values depend on the operating system or tool	Different tools and systems start from different values
Speed, loss and jitter (9)	rate, sload, dload, sloss, dloss, sinpkt, dinpkt, sjit, djit	Packets and bits per second, lost packets, and timing between packets	Floods and abnormal tools change the speed and timing
TCP handshake and window (7)	tcprtt, synack, ackdat, swin, dwin, stcpb, dtcpb	How the TCP connection was set up, and TCP window and sequence values	Scanners and attack tools often behave differently when connecting
Connection counts (8)	ct_srv_src, ct_state_ttl, ct_dst_ltm, ct_src_dport_ltm, ct_dst_sport_ltm, ct_dst_src_ltm, ct_src_ltm, ct_srv_dst	How many recent connections share the same address, port, service or state pattern	Scans and floods create many similar connections
Protocol and service (8)	proto, service, state, trans_depth, is_ftp_login, ct_ftp_cmd, ct_flw_http_mthd, is_sm_ips_ports	The kind of traffic: protocol, service and connection state	Some attack types target particular services

7.3 A Worked Comparison of Two Real Records

The two real records below come from the training file. They show that the difference between normal and attack traffic is spread over several columns at once, which is what the model learns to recognise.

Table 11: A normal record and a DoS attack record from the training file

Column	Normal record (id 95745)	DoS attack record (id 49000)
dur (seconds)	0.661656	0.000009
proto / service / state	tcp / http / FIN	encap / - / INT
spkts / dpkts	10 / 10	2 / 0
sbytes / dbytes	804 / 1330	200 / 0
sttl	62	254
label (the correct answer)	0 (normal)	1 (attack)

The attack record lasted a tiny fraction of a second, no packets came back, and its sttl was 254. The normal record is a web connection with traffic in both directions. A single column rarely separates the two classes on its own, because other normal records also have short durations or high sttl values (the sample records in Chapter 4 show a normal record with sttl 254). The model therefore combines many such clues.

7.4 Evidence That No Single Column Decides

Three pieces of evidence from the project run show this.

1. **Correlation with the label.** The strongest link of any numeric column with the label is only 0.693 (sttl). Even the best single column is therefore far from a perfect indicator. The next strongest values are ct_state_ttl 0.578, dload -0.394, ct_dst_sport_ltm 0.357 and dmean -0.342.
2. **Impurity importance in the Random Forest.** The importance values add up to 1 over all encoded columns. The highest, sttl, is only about 0.10, and the five highest columns (sttl, ct_state_ttl, dload, rate, dttl) together account for roughly one third of the total. The

remaining two thirds is spread over many other columns. (Values are read from the importance chart.)

3. **Permutation importance.** When the values of one column are shuffled on a test sample, the F1-score drops. The largest drop caused by any single column (service) is only about 0.016. If one column alone decided the result, shuffling it would destroy the performance. The small drops show that other columns compensate.

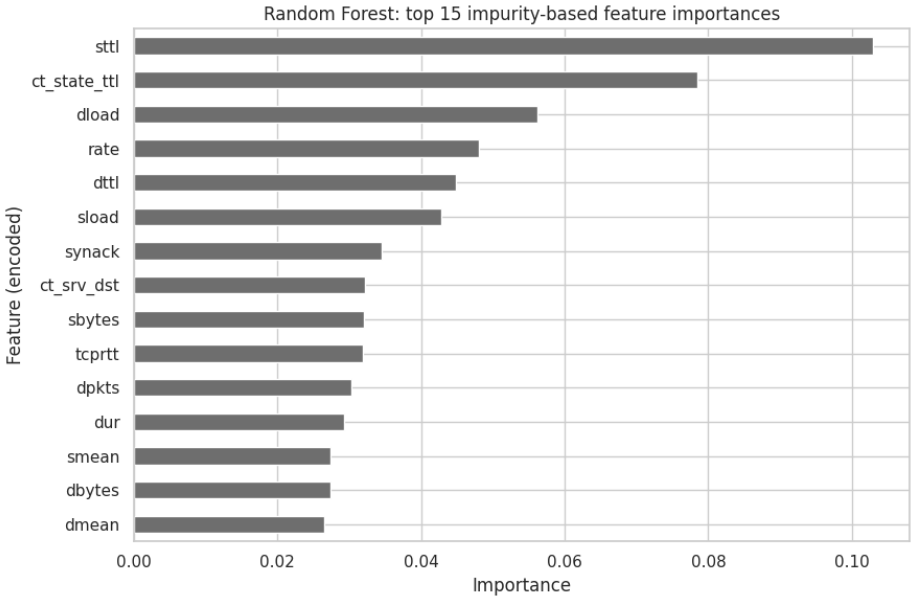


Figure 4: Random Forest impurity-based importance of the top 15 columns

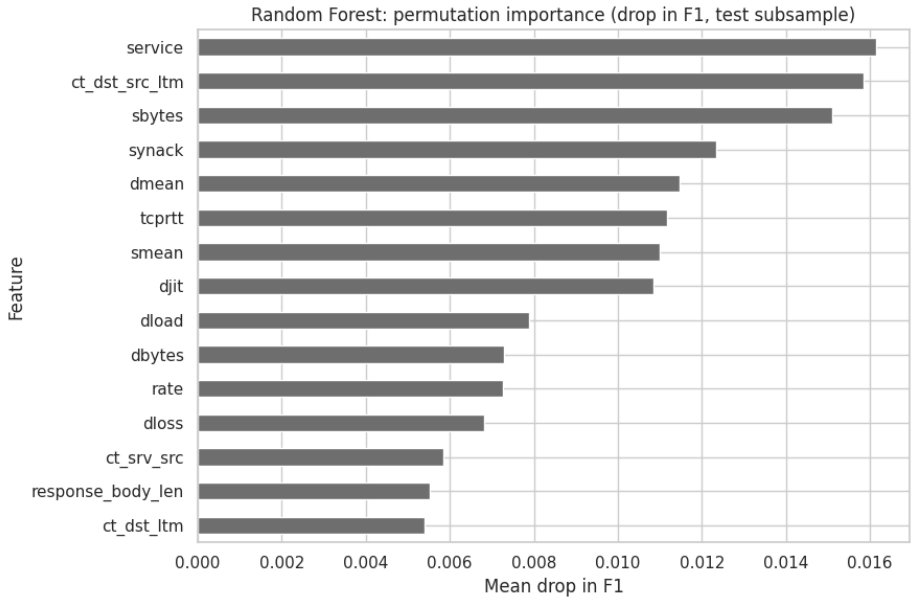


Figure 5: Random Forest permutation importance of the top 15 columns (drop in F1-score on a test sample)

7.5 The Most Influential Columns

Table 12: Columns that influence the Random Forest most, with the evidence

Column	Meaning	Evidence from the run
sttl	Time-to-live of packets sent by the source	Highest impurity importance (about 0.10); strongest correlation with the label (0.693)
ct_state_ttl	Count feature based on the connection state and the time-to-live values	Second in impurity importance; correlation 0.578
dload	Bits per second sent by the destination	Third in impurity importance; correlation -0.394
rate, dttl, sload	Packet rate; destination time-to-live; source bits per second	Fourth to sixth in impurity importance
service	The network service (http, dns, ftp and so on)	Highest permutation importance (F1 drop about 0.016)
ct_dst_src_ltm	Count of recent connections with the same source and destination addresses	Second in permutation importance
sbytes, synack, dmean, tcprtt	Bytes sent by the source; handshake timing; mean packet size of the destination; handshake round-trip time	Third to sixth in permutation importance

7.6 Limitations of the Importance Analysis

- The two importance methods do not agree on the order. Many columns carry overlapping information, so shuffling one column hurts little when others can replace it. Both rankings are therefore partial views.
- Importance and correlation show **association**, not cause. A high sttl does not cause an attack. The pattern may partly reflect how the attack traffic was generated in the laboratory. The dominance of sttl was also reported by Kasongo and Sun [2].
- These rankings describe this model and this dataset. They should not be assumed to hold on another network.
- I did not run an experiment that removes a column and re-trains the model, so I cannot say how much accuracy would be lost without any particular column. This is listed as future work.

8. Research Methodology

8.1 Research Design

The project follows a quantitative, experimental and comparative research design. It is quantitative because results are measured numbers (counts and rates), experimental because the same procedure is applied to every model under controlled conditions, and comparative because three models are compared against each other and against a baseline. The work was organised in phases similar to the CRISP-DM process model [7]: problem understanding, data understanding, data preparation, modelling, evaluation and reporting.

8.2 Phases of the Study

Table 13: Phases of the research methodology

Phase	Activity	Output
1. Problem understanding	Define the problem, objectives and research question	Chapters 1 and 2
2. Literature review	Select and study five research papers	Chapter 3
3. Dataset selection	Compare candidate datasets and select UNSW-NB15	Chapters 4 and 5
4. Data understanding	Load files, inspect rows, quality and class balance, explore patterns	Chapter 13
5. Data preparation	Clean data, separate features and target, build the pipeline	Chapters 9 and 10
6. Modelling	Train baseline and three models; tune hyperparameters	Chapters 6, 10 and 11
7. Evaluation	Test once on the official test file; analyse errors and columns	Chapters 7, 14 and 15
8. Reporting	State findings, limitations, conclusion and future work	Chapters 15 to 18

8.3 Data Collection

The study uses secondary data: an existing public dataset created by other researchers. No data was collected from people, no personal data was used, and no network traffic was captured or monitored by me. Both official files were used completely, so no sampling was done.

8.4 Variables

- **Independent variables:** the 42 input features, the type of model, and the hyperparameter values.
- **Dependent variable:** the predicted class (normal or attack), judged through the evaluation metrics.
- **Controlled variables:** the dataset files, the train/test split, the preprocessing steps, the random seed (42), the tuning method and the evaluation metrics, which are identical for all models.

8.5 Model Selection Approach

Models were chosen to cover three families from simple to stronger: a linear model (Logistic Regression), a single tree (Decision Tree) and an ensemble of trees (Random Forest). Random Forest was selected as the main model by design, because it handles mixed tabular features and non-linear patterns well. Whether it actually performs best was left for the results to decide. Testing the other models is what makes the choice evidence-based: without a comparison there would be no way to know whether the main model is better, or by how much.

8.6 Evaluation Approach

Models are tuned only on the training file and then judged once on the official test file. Because attack and normal records are unequal in number, accuracy alone is not trusted. Precision, recall, F1-score, false-positive rate, false-negative rate, ROC-AUC, PR-AUC, confusion matrices and detection rates per attack category are reported together.

8.7 Validity and Reliability

- **Internal validity:** preprocessing is fitted on training data only; the identifier, the attack name and the label are excluded from the inputs; the test file is used once.
- **Construct validity:** several metrics are used because no single number represents detection quality.
- **External validity:** limited, because only one dataset is used. This is acknowledged in Chapter 16.
- **Reliability:** a fixed random seed is used, library versions are recorded, and settings and results are saved to files so the run can be repeated.

8.8 Tools and Environment

Table 14: Tools and environment

Item	Details
Platform	Google Colab, Python 3
Libraries	pandas 2.2.3, numpy 2.1.3, scikit-learn 1.6.1, matplotlib, seaborn
Data storage	Google Drive folder, mounted in Colab
Code	Jupyter notebook UNSW_NB15_Intrusion_Detection.ipynb

8.9 Ethical Considerations

Only a public dataset released for research was used, with the creators cited. The purpose is defensive: improving the detection of malicious traffic. No attack tools were built or run, and no real network was scanned or attacked.

9. System Design

9.1 Overview

The system is a data-processing and modelling workflow built as one notebook. Its input is a table of network-flow records, and its outputs are predictions (normal or attack), evaluation tables and figures. It is organised in six layers.

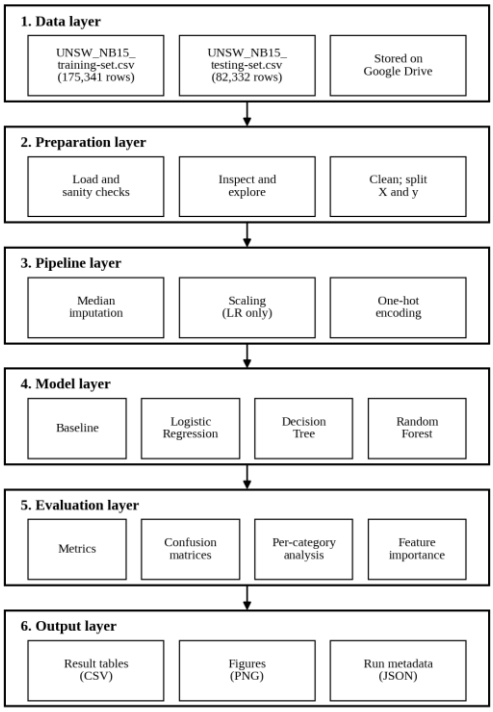


Figure 6: System design of the project in six layers

9.2 Components

Table 15: Components of the system

Layer	Component	Function
Data	Two CSV files on Google Drive	Provide the labelled records
Preparation	Loader, checks, cleaning, feature and target split	Read the files, verify the columns and labels, convert types, set infinite values to missing, separate X and y
Pipeline	Imputer, scaler, one-hot encoder	Fill missing numbers with the median, scale numbers for Logistic Regression only, turn categories into 0/1 columns
Model	Baseline, Logistic Regression, Decision Tree, Random Forest; GridSearchCV	Learn the pattern and tune hyperparameters with cross-validation
Evaluation	Metrics, confusion matrices, per-category analysis, feature importance	Measure and explain performance on the test file
Output	CSV tables, PNG figures, JSON metadata, zip file, saved model	Store the results for reporting and reuse

9.3 Input and Output Specification

- **Input:** one network-flow record with 42 features (39 numeric and 3 categorical).
- **Output:** a probability of attack and a predicted class (0 normal or 1 attack).

9.4 Design Decisions

Table 16: Main design decisions and their reasons

Decision	Reason
Use the official training and testing files	Keeps the test data separate and matches the literature
Exclude id, attack_cat and label from the inputs	id has no meaning; attack_cat and label reveal the answer
Put preprocessing in a pipeline	Each step learns only from training data, so nothing leaks from validation or test data
Scale features only for Logistic Regression	Linear models are sensitive to scale; tree models are not
Tune with stratified 5-fold cross-validation on the training file	Every fold keeps the same normal/attack ratio and the test file stays unseen
Use a majority-class baseline	Shows whether the models learned anything useful
Keep duplicate rows	Preserves the official files exactly as released; the effect is reported and discussed

10. Experimentation Design

10.1 Use of the Data

Table 17: How each file was used

Data	Used for	Not used for
Training file (175,341 rows)	Fitting preprocessing, cross-validation, tuning, training final models, random-split contrast	Final performance claims
Testing file (82,332 rows)	One final evaluation of each model; per-category analysis; permutation importance	Fitting, tuning or choosing any setting

10.2 Experimental Procedure

1. Load and verify the two files (column names and label values).
2. Inspect the data and record the class distribution, missing values, duplicates and category levels.
3. Clean the data and separate features and target; drop id, attack_cat and label from the inputs.
4. Build one pipeline per model.
5. Train the baseline.
6. For each of the three models, run grid search with 5-fold stratified cross-validation on the training file; select the combination with the best mean F1-score; refit it on the full training file.
7. Evaluate the baseline and the three final models once on the testing file.
8. Analyse errors by attack category, compare training, cross-validation and test scores, and compute feature importance.
9. Save all tables, figures and settings.

10.3 List of Experiments

Table 18: Experiments carried out

ID	Experiment	Purpose	Data used
E1	Baseline	Show what learning nothing scores	Train, then test
E2	Model comparison	Compare Logistic Regression, Decision Tree and Random Forest	Train for tuning; test for result
E3	Per-category detection	Find which attack types are missed	Test file with attack_cat
E4	Overfitting check	Compare cross-validation, training and test scores	Train and test
E5	Random-split contrast	See whether a random split is more optimistic than the official split	Training file split 80/20
E6	Feature importance	See which columns the model relies on	Fitted Random Forest; test subsample

10.4 Controls and Fairness

All models use the same data, the same fixed random seed, the same cross-validation folds, the same scoring rule (F1-score of the attack class) and the same evaluation code. The only differences are the models themselves and the preprocessing that each model needs (scaling for Logistic Regression only). The test file is never used for fitting, tuning or choosing between settings.

11. Parameters

This chapter lists all settings used in the experiments. In Machine Learning it is important to separate four ideas. **Parameters** are values learned from data during training, such as regression weights and tree split points. **Hyperparameters** are settings chosen before training, such as the maximum depth of a tree. **Training settings** are things like the random seed. The **evaluation threshold** is the probability cut-off (0.5 by default) that turns a score into the class attack.

11.1 General Settings

Table 19: General experiment settings

Setting	Value	Reason
Random seed	42	Repeatable results
Cross-validation	5-fold, stratified, shuffled	Stable estimates with the same class mix in each fold
Tuning score	F1-score of the attack class	Balances precision and recall
Test file use	Once, at the end	Keeps the test data unseen
Duplicate rows	Kept (DROP_TRAIN_DUPLICATES = False)	Official files used as released
Evaluation threshold	0.5 (default)	No threshold tuning was done
Random-split contrast	80% / 20% stratified split of the training file	Compare with the official test result
Permutation importance	10,000 test rows, 3 repeats, F1 scoring	Keeps the computation small

11.2 Preprocessing Parameters

Table 20: Preprocessing parameters

Step	Applied to	Parameter
Imputation	39 numeric features	Strategy: median (no values were missing, so it is a safeguard)
Scaling	39 numeric features, Logistic Regression only	StandardScaler (mean 0, standard deviation 1)
Encoding	proto, service, state	One-hot encoding; unknown categories ignored
Excluded columns	id, attack_cat, label	Not used as inputs

11.3 Hyperparameters Searched and Chosen

Table 21: Hyperparameter search grids and the values chosen by cross-validation

Model	Hyperparameter	Values tried	Chosen
Logistic Regression	C (inverse regularisation strength)	0.1, 1, 10	10
Decision Tree	max_depth	5, 10, 20, None	20
	min_samples_leaf	1, 5, 20	1
Random Forest	n_estimators	100	100
	max_depth	15, None	None (unlimited)
	min_samples_leaf	1, 5	1
	max_features	sqrt	sqrt

Fixed settings: Logistic Regression used a maximum of 2,000 iterations. All other options were the scikit-learn defaults, for example the Gini criterion for the trees.

Table 22: Tuning results and time

Model	Combinations x folds	Mean cross-validation F1	Tuning time (seconds)
Logistic Regression	3 x 5	0.9546	85.8
Decision Tree	12 x 5	0.9635	221.2
Random Forest	4 x 5	0.9703	1,076.5

11.4 Role of the Main Hyperparameters

- **C (Logistic Regression):** controls regularisation. A smaller value is a stronger penalty and a simpler model.
- **max_depth:** limits how many questions a tree can ask. A smaller value gives a simpler tree that is less likely to overfit.
- **min_samples_leaf:** the minimum number of records allowed in a leaf. A larger value gives a simpler tree.
- **n_estimators:** the number of trees in the forest.
- **max_features:** the number of features considered at each split; the square root of the total was used.

11.5 Evaluation Metrics and Formulas

In the formulas below, TP is the number of attacks correctly flagged, TN the number of normal flows correctly passed, FP the number of normal flows wrongly flagged (false alarms) and FN the number of attacks missed. The attack class is the positive class.

Table 23: Evaluation metrics

Metric	Formula	Meaning in intrusion detection
Accuracy	(TP + TN) / (TP + TN + FP + FN)	Share of all flows classified correctly
Precision	TP / (TP + FP)	Of the alerts raised, how many were real attacks
Recall	TP / (TP + FN)	Of the real attacks, how many were caught
F1-score	2 x Precision x Recall / (Precision + Recall)	Balance of precision and recall
False-positive rate	FP / (FP + TN)	Share of normal traffic wrongly flagged
False-negative rate	FN / (FN + TP)	Share of attacks missed
ROC-AUC	Area under the true-positive rate versus false-positive rate curve	Ranking quality over all thresholds
PR-AUC	Area under the precision versus recall curve	Ranking quality focused on the attack class

12. Implementation and Evidence

This chapter shows how the project was implemented in code and how the notebook itself provides evidence of the work.

12.1 Implementation Overview

The whole project is one Jupyter notebook run in Google Colab. It has 28 code cells that load the data, inspect it, clean it, build the pipeline, train and tune the models, evaluate them, analyse errors and save the results. A final part of the notebook uses the trained model on single records and audits the work. Appendix B lists the cell structure.

12.2 Libraries and Main Components

Table 24: Libraries and classes used

Library	Classes and functions used	Purpose
pandas, numpy	read_csv, DataFrame, value_counts, duplicated	Load, inspect and clean the tables
scikit-learn: preprocessing	SimpleImputer, StandardScaler, OneHotEncoder, ColumnTransformer, Pipeline	Prepare the features without leakage
scikit-learn: models	LogisticRegression, DecisionTreeClassifier, RandomForestClassifier, DummyClassifier	The three models and the baseline
scikit-learn: selection	StratifiedKFold, GridSearchCV, train_test_split	Cross-validation and tuning
scikit-learn: metrics	confusion_matrix, f1_score, roc_auc_score, classification_report, permutation_importance	Evaluation and importance
matplotlib, seaborn	Bar charts, heatmaps, curves	Figures

12.3 Key Code Excerpts

The pipeline builder gives every model the same preprocessing recipe. Scaling is added only for Logistic Regression.

```
def make_pipeline(model, scale):
    num_steps = [("imputer", SimpleImputer(strategy="median"))]
    if scale:
        num_steps.append(("scaler", StandardScaler()))
    pre = ColumnTransformer([
        ("num", Pipeline(num_steps), NUM_COLS),
        ("cat", OneHotEncoder(handle_unknown="ignore"), CAT_COLS),
    ])
    return Pipeline([("prep", pre), ("model", model)])
```

Tuning uses stratified cross-validation on the training file only. The test file does not appear in these lines.

```
cv = StratifiedKFold(n_splits=CV_FOLDS, shuffle=True, random_state=SEED)
gs = GridSearchCV(spec["pipe"], spec["grid"], cv=cv, scoring="f1",
                  n_jobs=spec["grid_jobs"], refit=True)
gs.fit(X_train, y_train)
```

Evaluation uses the test file once. The confusion counts are calculated from the predictions.

```
tn, fp, fn, tp = confusion_matrix(y, y_pred, labels=[0, 1]).ravel()
results = {name: evaluate(m, X_test, y_test) for name, m in
final_models.items()}
```

12.4 Using the Trained Model: Input to Output

After training, the model can be given one connection and returns an answer. The final cell of the notebook defines a function that takes a record (the 42 feature values), asks the trained Random Forest for the probability of attack, and applies the threshold 0.5.

```
def predict_record(record, model=best_model, threshold=0.5):
    row = pd.DataFrame([record]).reindex(columns=FEATURE_COLS)
    prob = float(model.predict_proba(row)[0, 1])
    return ("ATTACK" if prob >= threshold else "NORMAL"), prob
```

Table 25: Two ways of giving the input

Input option	What the user provides	What comes out
A: ROW_NUMBER	A number from 0 to 82,331 that selects a real record of the test file	The input features, the model's answer and probability, the true answer and the attack type, and whether the model was correct
B: my_record	A few feature values typed by hand; the pipeline fills the missing ones with typical training values	The model's answer and probability (a rough demonstration only)

The notebook also predicts ten random test records in a table and saves the trained model to a file so that it can be reused without training again. The model reads the 42 features. Turning raw network packets into those features needs a separate extraction step that is not part of this project.

12.5 Implementation Proof Cell

To show that the work was really carried out in the notebook, a proof cell was added at the end. Nothing in it is typed by hand: every line is recomputed from the variables and files of the session. Its sections are listed below.

Table 26: What the proof cell checks

Section	What it verifies
1 Environment	Python and library versions, random seed, run time
2 Data provenance	The real CSV files were used: size on disk, SHA-256 fingerprint, shape; raw tables were never modified
3 Leakage safety	The imputer medians, the scaler statistics and the encoder categories were learned from the training file only; the Decision Tree root holds exactly the training rows
4 What was trained	Number of cross-validation fits, chosen hyperparameters, number of trees, depth and leaves
5 Metrics recomputed	Confusion counts and metrics are recalculated by plain counting and compared with the reported table
6 Reproducibility	Re-training Logistic Regression and Decision Tree gives identical predictions
7 Saved files	Result tables and figures exist on disk and match the results in memory
8 Input to output	Live predictions on unseen test records
9 Summary	Number of checks passed, a fingerprint of the results table, and a saved text report

[Insert here a screenshot of the output of the proof cell, showing the PASS lines and the summary.]

12.6 Reproducibility

The random seed is fixed at 42 for splitting, cross-validation and the models. The library versions (pandas 2.2.3, numpy 2.1.3, scikit-learn 1.6.1) and all settings are saved in a metadata file. Another person who uses the same data files and library versions can repeat the run.

13. Data Analysis

This chapter reports the properties of the data observed before modelling. All numbers come from the project run.

13.1 Class Balance

In the training file 68.06% of the records are attacks, and in the testing file 55.06% are attacks. Normal traffic is therefore the minority class in training. The binary imbalance is moderate, but the attack categories are very unequal in size: Worms has only 130 training records while Generic has 40,000.

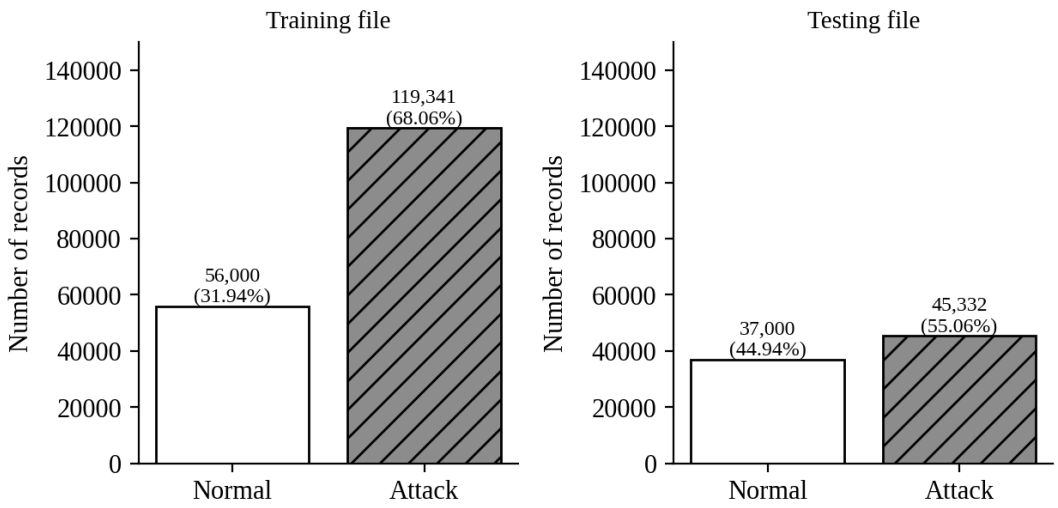


Figure 7: Normal and attack records in the training and testing files

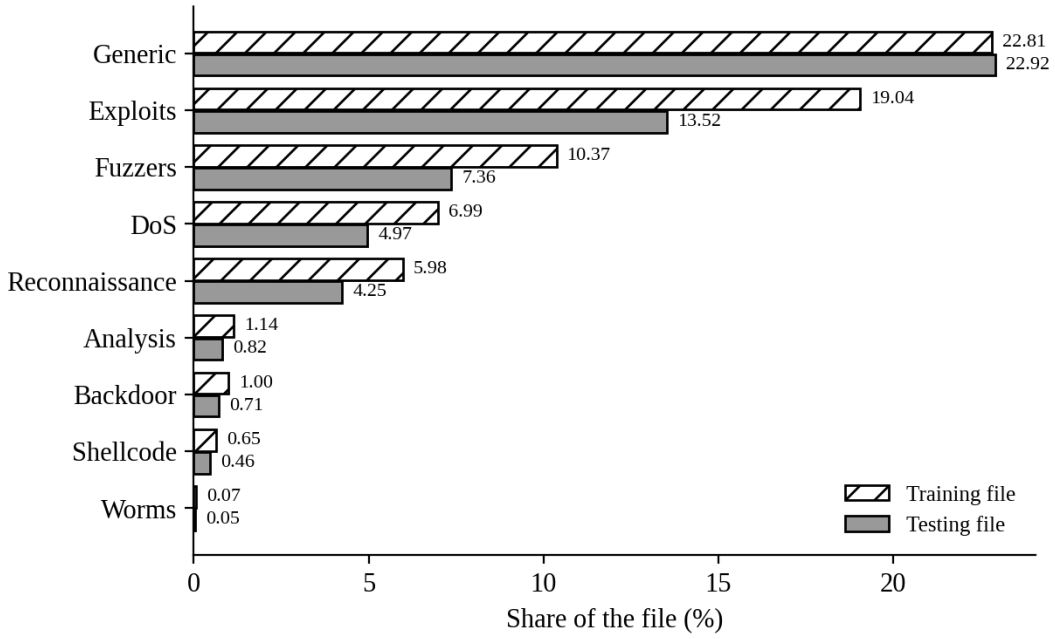


Figure 8: Share of each attack category in the training and testing files

13.2 Differences between the Training and Testing Files

The two files do not have the same traffic mix. The attack share falls from 68.06% to 55.06%, and some categories shrink noticeably, for example Exploits from 19.04% to 13.52% and Fuzzers from 10.37% to 7.36%. The testing file also contains two connection states (ACC and CLO) that never appear in the training file. A model trained on one file therefore meets slightly different traffic in the other. Data quality and duplicate counts are given in Section 4.8.

13.3 Columns Most Associated with the Label

Table 27: Numeric features with the strongest correlation with the label (training file)

Feature	Correlation with label	Feature	Correlation with label
sttl	0.693	rate	0.338

Feature	Correlation with label	Feature	Correlation with label
ct_state_ttl	0.578	swin	-0.334
dload	-0.394	dwin	-0.320
ct_dst_sport_ltm	0.357	ct_src_dport_ltm	0.306
dmean	-0.342	ct_dst_src_ltm	0.304

The feature sttl has the strongest link with the label. Correlation measures only straight-line association and does not show that the feature causes an attack. The strongest value, 0.693, also shows that no single column separates the classes perfectly (see Chapter 7).

14. Results

All results below were obtained on the official testing file (82,332 records: 37,000 normal and 45,332 attack). The attack class is the positive class.

14.1 Overall Model Comparison

Table 28: Performance of all models on the official test file

Model	Accuracy	Precision	Recall	F1	FPR	FNR	ROC-AUC	PR-AUC
Baseline	0.5506	0.5506	1.0000	0.7102	1.0000	0.0000	0.5000	0.5506
Logistic Regression	0.8110	0.7549	0.9725	0.8500	0.3869	0.0275	0.9562	0.9664
Decision Tree	0.8693	0.8300	0.9590	0.8899	0.2407	0.0410	0.8970	0.8656
Random Forest	0.8719	0.8188	0.9853	0.8944	0.2671	0.0147	0.9783	0.9813

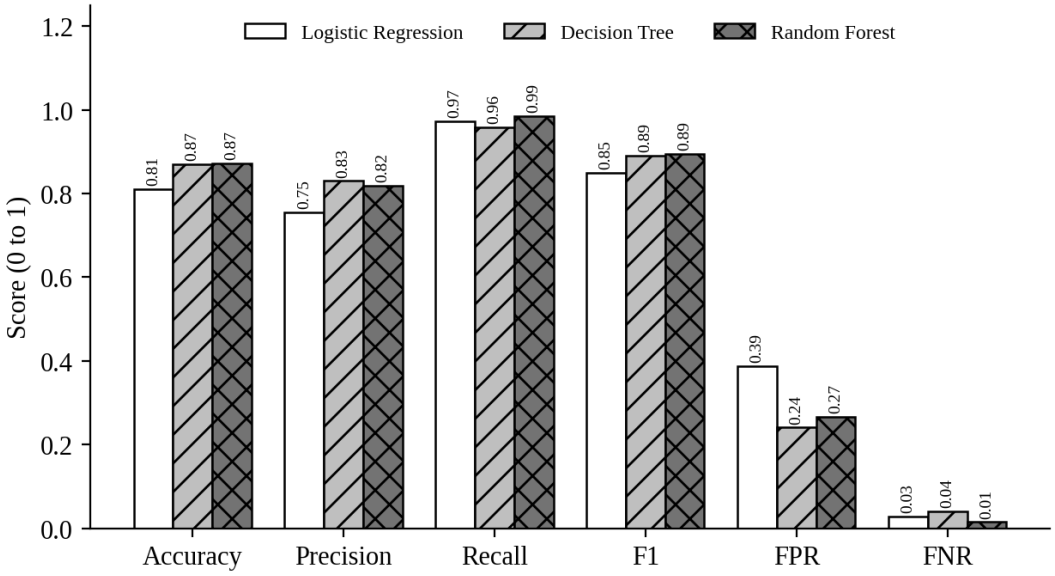


Figure 9: Key test-file metrics for the three models (lower is better for FPR and FNR)

14.2 Confusion Matrices

Table 29: Confusion counts on the test file

Model	TN (normal passed)	FP (false alarms)	FN (missed attacks)	TP (attacks caught)
Baseline	0	37,000	0	45,332
Logistic Regression	22,686	14,314	1,248	44,084
Decision Tree	28,094	8,906	1,857	43,475
Random Forest	27,118	9,882	668	44,664

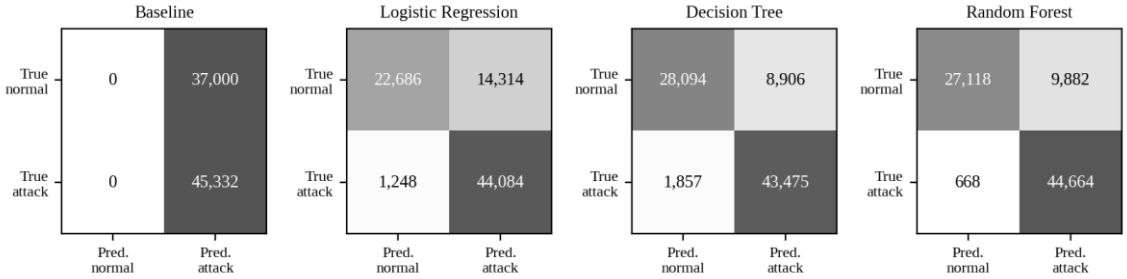


Figure 10: Confusion matrices on the official test file (rows are the true class, columns the predicted class)

As a check, the Random Forest values give precision = 44,664 / (44,664 + 9,882) = 0.8188, recall = 44,664 / 45,332 = 0.9853, false-positive rate = 9,882 / 37,000 = 0.2671 and F1-score = 2 x 0.8188 x 0.9853 / (0.8188 + 0.9853) = 0.8944, which match the results table.

14.3 Detection by Attack Category

Table 30: Detection rate per category on the test file (for Normal: share correctly passed)

Category	Test records	Logistic Regression	Decision Tree	Random Forest
Analysis	677	0.9143	0.9394	1.0000
Backdoor	583	0.9897	0.9931	1.0000
DoS	4,089	0.9790	0.9868	0.9993
Exploits	11,132	0.9429	0.9759	0.9958
Fuzzers	6,062	0.9301	0.7794	0.8987
Generic	18,871	0.9990	0.9956	1.0000
Normal (passed)	37,000	0.6131	0.7593	0.7329
Reconnaissance	3,496	0.9963	0.9854	0.9997
Shellcode	378	0.9868	0.9524	0.9921
Worms	44	0.9545	0.9773	1.0000

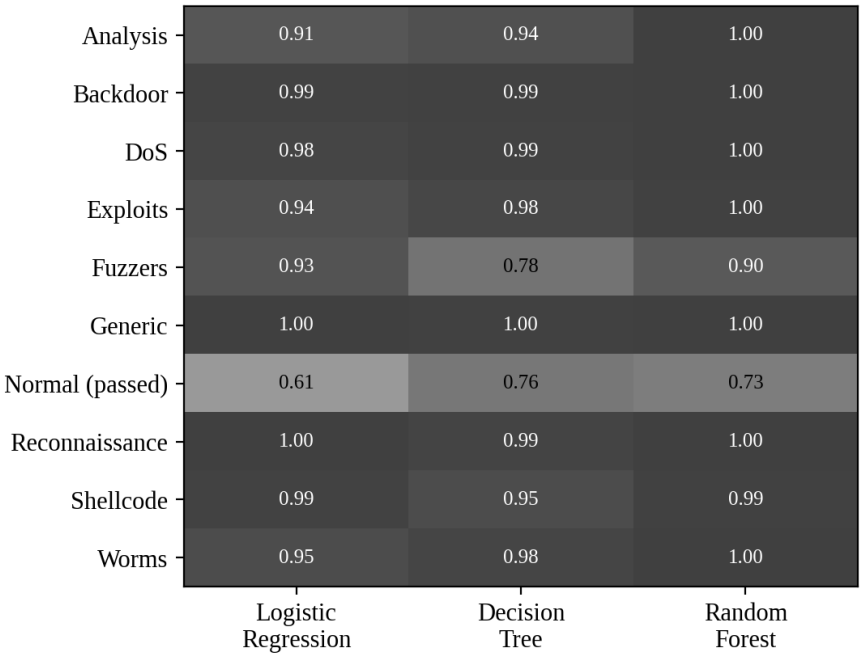


Figure 11: Detection rate per category (darker means higher)

14.4 Overfitting Check and Random-Split Contrast

Table 31: Training-side and test scores (F1-score)

Model	Mean CV F1	F1 on full training file	F1 on official test file	Training minus test
Logistic Regression	0.9546	0.9548	0.8500	0.1048
Decision Tree	0.9635	0.9853	0.8899	0.0954
Random Forest	0.9703	0.9987	0.8944	0.1043

In the random-split contrast, the tuned Random Forest was trained on a random 80% of the training file and tested on the remaining 20%. It reached an F1-score of **0.9702**, compared with **0.8944** on the official test file.

14.5 Feature Importance

The two importance charts are shown in Chapter 7. Impurity-based importance ranked sttl, ct_state_ttl, dload, rate, dttl and sload highest. Permutation importance ranked service, ct_dst_src_ltm, sbytes, synack, dmean and tcprtt highest.

15. Discussion and Findings

15.1 Findings

1. **All three models beat the baseline.** The baseline accuracy was 0.5506, against 0.8110, 0.8693 and 0.8719 for the models. The models learned real patterns. The baseline's F1-score of 0.7102 looks decent even though it simply calls everything an attack, which shows that F1-score alone can flatter a poor model.
2. **Random Forest was best overall, but narrowly.** It had the highest F1-score (0.8944), ROC-AUC (0.9783) and PR-AUC (0.9813) and the fewest missed attacks (668, which is

1.47% of attacks). Its F1 lead over the Decision Tree is only 0.0045 in a single run, so it should not be described as clearly better.

3. **False alarms are the main weakness.** Normal traffic was wrongly flagged at rates of 38.7% (Logistic Regression), 24.1% (Decision Tree) and 26.7% (Random Forest). For Random Forest this means 9,882 false alarms, and about 18.1% of its 54,546 alerts were false. In practice this would cause alert fatigue.
4. **The models show a trade-off.** Random Forest missed the fewest attacks, whereas the Decision Tree raised the fewest false alarms. Which is preferable depends on whether a missed attack or a false alarm costs more.
5. **Fuzzers were the hardest attack category.** They were detected 77.9% of the time by the Decision Tree, 89.9% by Random Forest and 93.0% by Logistic Regression, whereas most other categories were above 94%. Very small categories such as Worms (44 test records) show high detection, but these estimates rest on few records.
6. **Training-side scores are much higher than test scores.** The gap between training and test F1-score is about 0.095 to 0.105 for all three models, including the simple Logistic Regression. Because even the simplest model shows it, the gap is probably not caused by overfitting alone. A likely explanation is that the testing file differs from the training file (different class mix, unseen connection states, different traffic). Duplicate rows may also make cross-validation scores optimistic, because a duplicate can appear in both a training fold and a validation fold. This explanation has not been proven by a separate experiment.
7. **A random split gives an optimistic result.** The random split inside the training file gave an F1-score of 0.9702, far above the official test result of 0.8944. This agrees with the concern raised in the literature [5] that the way data is split changes reported performance.
8. **No single column decides.** The model combines many columns. The strongest correlation with the label is 0.693, the highest impurity importance is about 0.10, and shuffling the most important single column lowers F1 by only about 0.016. The influential columns include sttl, ct_state_ttl, dload, service, ct_dst_src_ltm and sbytes, but they act together.
9. **Feature importance depends on the method.** Impurity importance highlights sttl and ct_state_ttl, while permutation importance highlights service, ct_dst_src_ltm and sbytes, and sttl is not among its top 15. Many features carry overlapping information. Importance shows what the model uses, not what causes an attack.

15.2 Answers to the Sub-questions

Table 32: Answers to the research sub-questions

Sub-question	Answer from the results
SQ1: Do the models beat the baseline?	Yes. Accuracy 0.8110 to 0.8719 against 0.5506
SQ2: Which model balances detection and false alarms best?	Random Forest is best overall on F1-score and recall; Decision Tree has the lowest false-alarm rate. The difference in F1-score is small
SQ3: Which attack type is hardest?	Fuzzers (77.9% to 93.0% detected)
SQ4: Is a random split more optimistic?	Yes. F1-score 0.9702 on a random split against 0.8944 on the official test file
SQ5: Which columns matter, and does one decide alone?	sttl, ct_state_ttl, dload, service, ct_dst_src_ltm and sbytes matter most, but no single column decides alone

15.3 Relation to the Literature

The results agree with the earlier papers in general patterns, and not in exact numbers, because the experimental set-ups differ. As in Kasongo and Sun [2], the feature sttl is very influential in the impurity ranking and there is a clear gap between training-side and test scores. As in Ngo et al. [3], normal traffic is the weaker class in binary classification. As in Hossain et al. [5], a random split gives a more optimistic score than a separate test setting. The rare-class result of Ngo et al. concerns multiclass classification, where the exact attack name must be predicted. The detection rates in this report mean only "flagged as an attack", so the two measurements should not be compared directly.

16. Limitations

16.1 Data

- Only one dataset was used. It was generated in a laboratory in 2015 and the attacks are synthetic, so it may not represent today's networks.
- A large share of rows are duplicates (about 42.2% of the training file and 34.5% of the testing file), and 10.37% of the test records are identical to training records. The duplicates were kept, so some scores, especially cross-validation scores, may be optimistic.
- Some attack categories are very small (Worms 44 test records, Shellcode 378), so their detection rates are uncertain.

16.2 Method

- The experiment is offline: records are classified after they have been collected, and the models were not tested for real-time use on streaming traffic, with limited memory and strict time limits. The work is a benchmark experiment and not a production-ready detection system.
- The task is binary. The models do not name the attack type.
- The tuning grids were small to suit free Colab, and several chosen values lie at the edge of the grid (C = 10, tree depth 20, minimum leaf size 1, unlimited forest depth), so a wider search might choose differently.
- Each result comes from a single run with no confidence intervals, so small differences, such as the 0.0045 difference in F1-score between Random Forest and Decision Tree, are not proven.
- The probability threshold was left at 0.5 and was not tuned.

16.3 Results and Generalisation

- False-alarm rates of 24% to 39% of normal traffic would be too high for practical use.
- The models were not tested on other datasets or on attack types absent from training. The reviewed papers [4], [5] report large drops across datasets, so these scores should not be assumed to transfer.
- Feature importance shows association, not cause, and the two importance methods disagreed. No experiment removed a column and re-trained the model.
- Traffic and attacks change over time (concept drift), and this was not studied.

17. Conclusion

This project compared Logistic Regression, Decision Tree and Random Forest for classifying network flows from the official UNSW-NB15 training and testing files as normal or attack. The models learn from 175,341 labelled examples: they see 42 feature columns together with the correct answer in the label column, and they learn patterns that link the measurements to the answer. When predicting, they see only the 42 features and output a probability of attack. No single column decides. Columns such as sttl, ct_state_ttl, dload, service, ct_dst_src_ltm and sbytes influence the decision most, but they act together.

After tuning with 5-fold stratified cross-validation on the training file only, all three models clearly beat the majority-class baseline, whose accuracy was 0.5506. On the official test file, Random Forest performed best overall, with an accuracy of 0.8719, an F1-score of 0.8944 and a recall of 0.9853, missing only 668 of 45,332 attacks. The Decision Tree was close behind (F1-score 0.8899) and raised fewer false alarms (false-positive rate 0.2407 against 0.2671), and Logistic Regression was the weakest (F1-score 0.8500, false-positive rate 0.3869).

The main weaknesses were false alarms on normal traffic and the Fuzzers category, which was detected 77.9% to 93.0% of the time. A gap of about 0.10 between training-side and test F1-scores appeared even for Logistic Regression, which suggests that the training and testing files differ, and a random split inside the training file gave a much more optimistic F1-score (0.9702) than the official test (0.8944). The data also contained many duplicate rows.

These results come from one 2015 laboratory dataset, so they do not prove how the models would perform on other data or on real networks. The project achieved its objectives: the dataset was inspected, a leakage-safe pipeline was built, three models and a baseline were compared fairly, errors were analysed, the decision process was explained, and the limitations were stated.

18. Future Work

- Remove duplicate rows from the training file, repeat the experiments and compare the results with those reported here.
- Remove the most influential columns (for example sttl) one at a time, re-train the model and measure how much performance is lost, to test how much the model depends on them.
- Widen the hyperparameter grids and report confidence intervals from repeated runs.
- Choose the probability threshold on a validation set to balance false alarms against missed attacks.
- Add gradient boosting models such as XGBoost and compare them with Random Forest.
- Extend the task to multiclass classification so that the attack type is also predicted.
- Test the trained models on another dataset to measure generalisation.

References

[1] N. Moustafa and J. Slay, "UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in 2015 Military Communications and Information Systems Conference (MilCIS), 2015, pp. 1-6. doi: 10.1109/MilCIS.2015.7348942.

[2] S. M. Kasongo and Y. Sun, "Performance analysis of intrusion detection systems using a feature selection method on the UNSW-NB15 dataset," Journal of Big Data, vol. 7, article 105, 2020. doi: 10.1186/s40537-020-00379-6.

[3] V.-D. Ngo, T.-C. Vuong, T. V. Luong and H. Tran, "Machine Learning-Based Intrusion Detection: Feature Selection versus Feature Extraction," arXiv:2307.01570, 2023 (preprint). https://arxiv.org/abs/2307.01570

[4] S. Layeghy and M. Portmann, "On Generalisability of Machine Learning-based Network Intrusion Detection Systems," arXiv:2205.04112, 2022 (preprint). https://arxiv.org/abs/2205.04112

[5] M. Z. Hossain, M. A. R. Khan, M. R. Islam, S. M. S. Islam and T. Gedeon, "Assessing Generalisation Capability of Machine Learning Models for Intrusion Detection," arXiv:2605.04407, v1, May 2026 (preprint). https://arxiv.org/abs/2605.04407

[6] N. Moustafa and J. Slay, "The evaluation of network anomaly detection systems: Statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set," Information Security Journal: A Global Perspective, vol. 25, no. 1-3, pp. 18-31, 2016.

[7] P. Chapman et al., "CRISP-DM 1.0: Step-by-step data mining guide," CRISP-DM Consortium, 2000.

[8] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[9] W. McKinney, "Data structures for statistical computing in Python," in Proceedings of the 9th Python in Science Conference, 2010, pp. 56-61.

[10] J. D. Hunter, "Matplotlib: A 2D graphics environment," Computing in Science and Engineering, vol. 9, no. 3, pp. 90-95, 2007.

[11] UNSW Canberra Cyber Range Lab, "The UNSW-NB15 dataset," https://research.unsw.edu.au/projects/unsw-nb15-dataset (dataset page, accessed during the project).

Appendix: Feature Dictionary

Summary of the official feature descriptions. The authoritative text is in the features file on the dataset page.

Table 33: Feature dictionary

Column(s)	Meaning	Use in this project
id	Row identifier	Excluded
dur	Total duration of the flow in seconds	Feature
proto	Transaction protocol, for example tcp or udp	Feature (categorical)
service	Network service such as http, dns or ftp; "-" means none	Feature (categorical)
state	State of the connection, for example FIN, CON or INT	Feature (categorical)
spkts, dpkts	Packets from source to destination and from destination to source	Feature
sbytes, dbytes	Bytes from source to destination and from destination to source	Feature
rate	Packets per second in the flow	Feature
sttl, dttl	Time-to-live values of source and destination packets	Feature
sload, dload	Source and destination bits per second	Feature
sloss, dloss	Packets retransmitted or dropped (source and destination)	Feature
sinpkt, dinpkt	Mean time between packets (source and destination)	Feature
sjit, djit	Jitter (variation in packet timing) for source and destination	Feature
swin, dwin	TCP window advertisement values	Feature
stcpb, dtcpb	TCP base sequence numbers	Feature
tcprtt, synack, ackdat	TCP handshake round-trip time and its two parts	Feature
smean, dmean	Mean packet size sent by source and destination	Feature
trans_depth	Depth of HTTP request/response pipelining	Feature
response_body_len	Size of the data returned by an HTTP server	Feature
ct_srv_src, ct_state_ttl, ct_dst_ltm, ct_src_dport_ltm, ct_dst_sport_ltm, ct_dst_src_ltm, ct_src_ltm, ct_srv_dst	Connection-count features: counts of recent connections sharing the same address, port, service or state/TTL pattern	Feature
is_ftp_login, ct_ftp_cmd, ct_flw_http_mthd	FTP login flag, FTP command count, HTTP method count	Feature
is_sm_ips_ports	1 if source and destination IP and port are equal, otherwise 0	Feature
attack_cat	Name of the traffic category	Excluded from inputs; used for error analysis
label	0 = normal, 1 = attack	Target

GitHub Link:
https://github.com/sayalisontakke/network_intrusion_detection_machinelearning_miniproject